

The Startup Idea Tracker

Phase 1: The Modern Environment Setup

You must avoid installing libraries globally to keep your system clean. Instead, you will create an isolated sandbox for this project.

- **Initialize with uv:** Use the ultra-fast, Rust-based package manager **uv**. Run `uv init startup-api` to create your standard project structure.
- **Create the Environment:** Verify your **uv** version and initialize the virtual environment inside your project folder.
- **Add the Framework:** Install **FastAPI** using the command `uv add fastapi`.

Phase 2: Building the RESTful API

An API is a set of rules that lets software modules communicate. You will build a **RESTful** system that follows principles like **Statelessness** and **Client-server decoupling**.

Your API must handle "Startup Ideas" with these endpoints:

- **GET /ideas:** Retrieve a list of all submitted startup ideas.
- **POST /ideas:** Add a new startup idea (Include: Name, Industry, and Budget).
- **PUT /ideas/{id}:** Update an existing idea's details as it evolves.
- **DELETE /ideas/{id}:** Remove an idea that didn't make the cut.

Phase 3: Testing with Automatic Docs

One of the main reasons **FastAPI** is famous is because it provides an interactive **Swagger UI** for testing.

- Launch your server and navigate to the `/docs` endpoint in your browser.
- **The Goal:** Successfully execute a **POST** to add a "Smart Coffee Machine" idea, then use **GET** to verify it was saved.

Phase 4: The "Professional" Finish

- **Git Practice:** Initialize a Git repository in your folder.
- **Push to GitHub:** Push your `main.py` and configuration files to a public repository.
- **Update Google sheet:** Put your github repo link in the google sheet session2_task tab.

Learning Resources:

- 1) FastAPI playlist by CampuX in Youtube.
- 2) FastAPI Course by Coursera (paid but you can get 90% off by applying for FA)

CheatSheet:

1. Installing uv (The Ultra-Fast Manager)

uv is the modern, Rust-based tool that replaces older, slower package managers.

Platform	Command
Windows	<code>powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 iex"</code>
Linux	<code>curl -LsSf https://astral.sh/uv/install.sh sh</code>

Note: After installation, verify with `uv --version` to ensure it is in your system path.

2. Project Initialization

Use these commands to create an isolated sandbox for your project, ensuring your global system remains clean.

1. Create a project folder and move into it
`mkdir startup-api && cd startup-api`

2. Initialize a standard project structure
`uv init`

3. Create and activate a virtual environment
`uv venv`

Windows Activation: `.\.venv\Scripts\activate`

Linux Activation: `source .venv/bin/activate`

3. Managing Dependencies

FastAPI is a modern framework known for its high performance and speed of development.

```
# Install FastAPI
uv add fastapi
```

```
# Install Uvicorn (The server required to run your API)
uv add uvicorn
```

4. Running the FastAPI Application

Once your `main.py` is ready, use the following commands to launch the server.

- **Standard Run:** `uv run uvicorn main:app`
- **Development Mode (Auto-Reload):** `uv run uvicorn main:app --reload`
 - The `--reload` flag ensures the server restarts every time you save a change, saving you from manual re-compilation.

5. Testing & Documentation

One of the most powerful features of FastAPI is its **Automatic Docs**.

- **Interactive Swagger UI:** Open your browser and go to `http://127.0.0.1:8000/docs`.
- **Alternative Docs:** Access Redoc at `http://127.0.0.1:8000/redoc`.

6. Git & Submission (The Final Step)

To lock in your work and share your "Win" with the club:

```
# Initialize local repository
git init
```

```
# Add and commit your files
git add .
git commit -m "Initial commit: The Hacker Setup"
```

```
# Connect to your GitHub and push
git remote add origin <your-repo-link>
git branch -M main
git push -u origin main
```